

Software Testing and Quality Assurance

Module 2: Testing Techniques

Static testing

Static testing is a software testing technique that examines the code, design, documentation, and other project artifacts—without actually executing the program or any of its components. In contrast, dynamic testing involves running the application to detect errors during execution. Static testing is designed to catch defects early in the development lifecycle, when they are easier and less costly to fix. Over 60% of program errors can be detected with static testing, often with the advantage of pinpointing the exact location of each error, unlike dynamic testing.

Static testing can be applied at every stage of the software development life cycle (SDLC), including requirements, design, test planning, coding, documentation, and maintenance procedures. It checks products for conformance to specifications and standards at each stage but does not demonstrate that the software is operational. As a cost-effective technique, static testing helps catch errors early in the process, preventing them from becoming bugs in later phases.

Static testing techniques help to produce a better product. Given below are some of the benefits of adopting a static testing approach:

1. As defects are found and fixed, the quality of the product increases.
2. A more technically correct base is available for each new phase of development.
3. The overall software life cycle cost is lower, since defects are found early and are easier and less expensive to fix.
4. The effectiveness of the dynamic test activity is increased and less time needs to be devoted for testing the product.
5. Immediate evaluation and feedback to the author from his/her peers which will bring about improvements in the quality of future products.

The objectives of static testing can be summarized as follows:

1. To identify errors in any phase of SDLC as early as possible
2. To verify that the components of software are in conformance with its requirements
3. To provide information for project monitoring
4. To improve the software quality and increase productivity

Types of static testing

Static testing can be categorized into the following types:

1. Software inspections
2. Walkthroughs
3. Technical reviews

Inspection:

These can be used to tackle software quality problems because they allow the detection and removal of defects after each phase of the software development process. The inspection process may be applied to any product or partial product of the software development process, including requirements, design and code, project management plan, SQA plan, software configuration plan (SCM plan), risk management plan, test cases, user manual, etc. Inspections are embedded in the process of developing products and are done in the early stages of each product's development.

This process does not require executable code or test cases. With inspection, bugs can be found on infrequently executed paths that are not likely to be included in test cases. Software inspection does not execute the code, so it is machine-independent, requires no target system resources or changes to the program's operational behaviour, and can be used much before the target hardware is available for dynamic testing purposes.

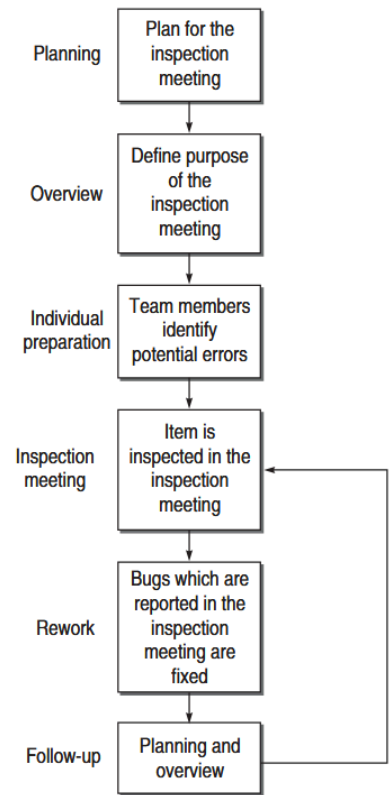
Inspection team: For the inspection process, a minimum of the following four are required.

- Author/owner/producer. A programmer or designer responsible for producing the program or document. He is also responsible for fixing defects discovered during the inspection process.
- Inspector. A peer member of the team, i.e. he is not a manager or supervisor. He is not directly related to the product under inspection and may be concerned with some other product. He finds errors, omissions, and inconsistencies in programs and documents.

- Moderator. A team member who manages the whole inspection process. He schedules, leads, and controls the inspection session. He is the key person with the responsibility of planning and successful execution of the inspection.
- Recorder. One who records all the results of the inspection meeting.

Inspection process: A general inspection process has the following stages:

- Planning: The product to be inspected is identified. A moderator is assigned. The objective of the inspection is stated, i.e. whether the inspection is to be conducted for defect detection or something else. The aim is to define an objective for the meeting so that the effort spent in inspections is properly utilized. During planning, the moderator performs the following activities:
- Overview. In this stage, the inspection team is provided with the background information for inspection. The author presents the rationale for the product, its relationship to the rest of the products being developed, its function and intended use, and the approach used to develop it. This information is necessary for the inspection team to perform a successful inspection.
- Individual preparation. After the overview, the reviewers individually prepare themselves for the inspection process by studying the documents provided to them in the overview session. They point out potential errors or problems found and record them in a log. This log is then submitted to the moderator. The moderator compiles the logs of different members and gives a copy of this compiled list to the author of the inspected item. The moderator reviews the logs submitted by each inspector to determine whether the team is adequately prepared. After this, the compiled log file is submitted to the author.
- Inspection meeting. The basic goal of the inspection meeting is to uncover any bug in the item. However, no effort is made in the meeting to fix the bug. It means that bugs are only being notified to the author, which he will fix later. If there is any clarification regarding these bugs, then it should be asked or discussed with other members during the meeting.
- Rework. The summary list of the bugs that arise during the inspection meeting needs to be reworked by the author. The author fixes all these bugs and reports back to the moderator.
- Follow-up. It is the responsibility of the moderator to check that all the bugs found in the last meeting have been addressed and fixed. He prepares a report and ascertains that all issues have been resolved. The document is then approved for release. If this is not the case, then the unresolved issues are mentioned in a report and another inspection meeting is called by the moderator.



Benefits of inspection process

- Bug reduction. The number of bugs is reduced through the inspection process. Based on the experiences of previous inspections, analysis can be made for future inspections or projects, thereby preventing the bugs which have appeared earlier. Programmers must understand why bugs appear and what can be done to avoid them in future.
- Productivity. Since all phases of SDLC may be inspected without waiting for code development and its execution, the cost of finding bugs decreases, resulting in an increase in productivity. Moreover, the errors are found at their exact places, therefore reducing the need for dynamic testing and debugging.
- Real-time feedback to software engineers. Developers find out the type of mistakes they make and what is the error density. Since they get this feedback in the early stages of development, they may improve their capability. Thus, inspections benefit software engineers/ developers in the sense that they can recognize their weakness and improve accordingly, which in turn benefits the cost of the project.
- Quality improvement. The direct consequence of testing is improvement in the quality of software. The direct consequence of static testing also results in the improvement of quality of the final product. Inspections help to improve the quality by checking the standard compliance, modularity, clarity, and simplicity.
- Process improvement. There is always scope of learning from the results of one process. An analysis of why the errors occurred or the frequent places where the errors occurred can be done by the inspection team members. The analysis results can then be used to improve the inspection process so that the current as well as future

projects can benefit. Discussed below are the issues of process improvement.

Effectiveness of inspection process

The effectiveness of the inspection process lies in its rate of inspection. The rate of inspection refers to how much evaluation of an item has been done by the team. If the rate is very fast, it means the coverage of items to be evaluated is high. Similarly if the rate is too slow, it means that the coverage is not much. A balanced rate of inspection should be followed so that it concentrates on a reasonable number of defects.

$$\text{Error detection efficiency} = \frac{\text{Error found by an inspection}}{\text{Total errors in the item before inspection}} \times 100$$

Review

A technical review is intended to evaluate the software in the light of development standards, guidelines, and specifications and to provide the management with evidence that the development process is being carried out according to the stated objectives. A review is similar to an inspection or walkthrough, except that the review team also includes management. Therefore, it is considered a higher-level technique as compared to inspection or walkthrough.

A technical review team is generally composed of management-level representatives and project management. Review agendas should focus less on technical issues and more on oversight than an inspection. The purpose is to evaluate the system relative to specifications and standards, recording defects and deficiencies. The moderator should gather and distribute the documentation to all team members for examination before the review. He should also prepare a set of indicators to measure the following points:

- Appropriateness of the problem definition and requirements
- Adequacy of all underlying assumptions
- Adherence to standards
- Consistency
- Completeness
- Documentation

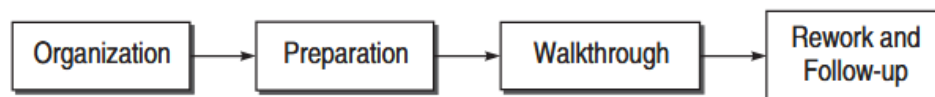
The moderator may also prepare a checklist to help the team focus on the key points. The result of the review should be a document recording the events of the meeting, deficiencies identified, and review team recommendations. Appropriate actions should then be taken to correct any deficiencies and address all recommendations.

Walkthrough

It is a less formal and less rigorous technique as compared to inspection. The common term used for static testing is inspection but it is a very formal process. If you want to go for a less formal process having no bars of organized meeting, then walkthroughs are a good option. A typical structured walkthrough team consists of the following members:

- Coordinator: Organizes, moderates, and follows up the walkthrough activities.
- Presenter/developer: Introduces the item to be inspected. This member is optional.
- Scribe/recorder: Notes down the defects found and suggestions proposed by the members.
- Reviewer/tester: Finds the defects in the item.
- Maintenance oracle: Focuses on long-term implications and future maintenance of the project.
- Standards bearer: Assesses adherence to standards.
- User representative/accreditation agent: Reflects the needs and concerns of the user.

Walkthroughs differ significantly from inspections. A walkthrough is less formal, has fewer steps and does not use a checklist to guide or a written report to document the team's work. The walkthrough should have a follow-up process similar to that described in the inspection process. The steps of a walkthrough process are shown below.



Dynamic testing

Dynamic Testing is a type of Software Testing that is performed to analyze the dynamic behavior of the code. It includes the testing of the software for the input values and output values that are analyzed. It is performed to describe the dynamic behavior of code. It refers to the observation of the physical response from the system to variables that are not constant and change with time.

Objectives of Dynamic Testing

- Find errors and bugs: Through comprehensive testing, find and expose flaws, faults, or defects in the software code and its functionality so that they can be fixed as soon as possible.
- Verify the behavior of the system: Verify that the software operates as expected and complies with company requirements, industry or regulatory standards, user expectations, and any applicable business regulations.
- Assessing Performance: To make sure the software satisfies performance requirements, evaluate its performance by monitoring reaction times, throughput, and use of resources under various scenarios.
- Assure Trustworthiness: Examine the software's dependability by determining how well it performs regularly under typical operating conditions, free of unexpected faults or crashes.
- Accuracy of Test Data: Verify the precision and consistency of the data handled by the software to guarantee reliable and uniform information handling.
- Assess Scalability: Examine whether the application can grow to handle more users, workloads, or data volumes without seeing an obvious decline in performance.

Advantages of Dynamic Testing

- Disclosure of Difficult and Complex Defects: It discloses very difficult and complex defects.
- Improvement in Software Quality: It increases the quality of the software product or application being tested.
- Security Threat Detection: Dynamic testing detects security threats and ensure the better secure application.
- Early-Stage Functionality Testing: It can be used to test the functionality of the software at the early stages of development.
- Ease of Implementation: It is easy to implement and does not require any special tools or expertise.
- Testing with Different Inputs, Data Sets, and User Profiles: It can be used to test the software with different input values, data sets and user profiles.
- Functionality and Performance Testing: It can be used to test the functionality of the code and performance of the code.

Disadvantages of Dynamic Testing

- Time-Consuming Process: It is a time consuming process as in dynamic testing whole code is executed.
- Increased Budget: It increases the budget of the software as dynamic testing is costly.
- Resource Intensive: Dynamic testing may require more resources than static testing.
- Less Effective in Some Cases: Dynamic testing may be less effective than static testing in some cases.
- Incomplete Test Scenario Coverage: It is difficult to cover all the test scenarios.
- Difficulty in Root Cause Analysis: It is difficult to find out the root cause of the defects.

White box testing

White-box testing is another effective testing technique in dynamic testing. It is also known as glass-box testing, as everything that is required to implement the software is visible. The entire design, structure, and code of the software have to be studied for this type of testing. It is obvious that the developer is very close to this type of testing. Often, developers use white-box testing techniques to test their own design and code. This testing is also known as structural or development testing.

In white-box testing, structure means the logic of the program which has been implemented in the language code. The intention is to test this logic so that required results or functionalities can be achieved. Thus, white-box testing ensures that the internal parts of the software are adequately tested.

Need of white box testing

1. White box testing techniques are used for testing the module for initial stage testing. Since test cases for black box testing cannot be executed until the code is produced and checked using white-box testing techniques, white box testing is not an alternative but an essential stage.
2. Since white-box testing is complementary to black-box testing, there are categories of bugs which can be revealed by white-box testing, but not through black-box testing.
3. We often believe that a logical path is not likely to be executed when, in fact, it may be executed on a regular basis. White-box testing explores these paths too.
4. Some typographical errors are not observed and go undetected and are not covered by black-box testing techniques. White-box testing techniques help detect these errors.

Basis path testing

Basis path testing is the oldest structural testing technique. The technique is based on the control structure of the program. Based on the control structure, a flow graph is prepared and all the possible paths can be covered and executed during testing. Path coverage is a more general criterion as compared to other coverage criteria and useful for detecting more errors. But the problem with path criteria is that programs that contain loops may have an infinite number of possible paths and it is not practical to test all the paths. Some criteria should be devised such that selected paths are executed for maximum coverage of logic. Basis path testing is the technique of selecting the paths that provide a basis set of execution paths through the program.

The guidelines for effectiveness of path testing are discussed below:

1. Path testing is based on the control structure of the program for which flow graph is prepared.
2. Path testing requires complete knowledge of the program's structure.
3. Path testing is closer to the developer and used by him to test his module.
4. The effectiveness of path testing gets reduced with the increase in size of software under test.
5. Choose enough paths in a program such that maximum logic coverage is achieved.

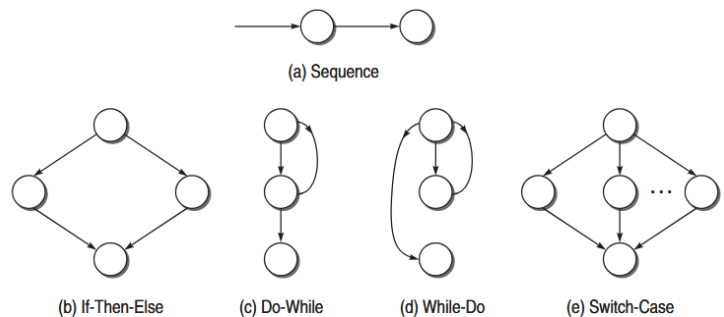
Control flow graph

The control flow graph is a graphical representation of the control structure of a program. Flow graphs can be prepared as a directed graph. A directed graph (V, E) consists of a set of vertices V and a set of edges E that are ordered pairs of elements of V . Based on the concepts of directed graph, following notations are used for a flow graph:

- Node. It represents one or more procedural statements. The nodes are denoted by a circle. These are numbered or labeled.
- Edges or links. They represent the flow of control in a program. This is denoted by an arrow on the edge. An edge must terminate at a node.
- Decision node. A node with more than one arrow leaving it is called a decision node.
- Junction node. A node with more than one arrow entering it is called a junction.
- Regions. Areas bounded by edges and nodes are called regions. When counting the regions, the area outside the graph is also considered a region.

Flow Graph Notations:

Since a flow graph is prepared on the basis of control structure of a program, some fundamental graphical notations are shown below for basic programming constructs. Using the above notations, a flow graph can be constructed. Sequential statements having no conditions or loops can be merged in a single node. That is why the flow graph is also known as decision-to-decision graph or DD graph.



Path Testing

- Path. A path through a program is a sequence of instructions or statements that starts at an entry, junction, or decision and ends at another, or possibly the same, junction, decision, or exit. A path may go through several junctions, processes, or decisions, one or more times.
- Segment. Paths consist of segments. The smallest segment is a link, that is, a single process that lies between two nodes. A direct connection between two nodes, as in an unconditional GOTO, is also called a process by convention, even though no actual processing takes place.
- Path segment. A path segment is a succession of consecutive links that belongs to some path.
- Length of a path. The length of a path is measured by the number of links in it and not by the number of instructions or statements executed along the path. An alternative way to measure the length of a path is by the number of nodes traversed. This method has some analytical and theoretical benefits. If programs are assumed to have an entry and an exit node, then the number of links traversed is just one less than the number of nodes traversed.
- Independent path. An independent path is any path through the graph that introduces at least one new set of processing statements or new conditions. An independent path must move along at least one edge that has not been traversed before the path is defined.

Cyclomatic Complexity:

Loop testing

Loop testing can be viewed as an extension to branch coverage. Loops are important in the software from the testing viewpoint. If loops are not tested properly, bugs can go undetected. This is the reason that loops are covered in this section exclusively. Loop testing can be done effectively while performing development testing (unit testing by the developer) on a module. Sufficient test cases should be designed to test every loop thoroughly. There are four different kinds of loops. How each kind of loop is tested, is discussed below.

Simple loops. Simple loops mean, we have a single loop in the flow, as shown in Fig. 5.9.

Check whether you can bypass the loop or not. If the test case for bypassing the loop is executed and, still you enter inside the loop, it means that there is a bug. Check whether the loop control variable is negative.

Write one test case that executes the statements inside the loop.

Write test cases for a typical number of iterations through the loop.

Write test cases for checking the boundary values of the maximum and minimum number of iterations defined (say min and max) in the loop. It means we should test for min, min+1, min-1, max-1, max, and max+1 number of iterations through the loop.

Nested loops. When two or more loops are embedded, it is called a nested loop, as shown in Fig. 5.10. If we have nested loops in the program, it becomes difficult to test. If we adopt the approach of simple tests to test the nested loops, then the number of possible test cases grows geometrically. Thus, the strategy is to start with the innermost loops while holding outer loops to their minimum values. Continue this outward in this manner until all loops have been covered.

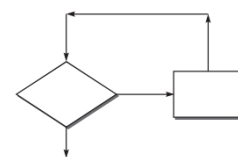


Figure 5.9 (a)

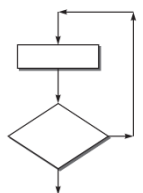
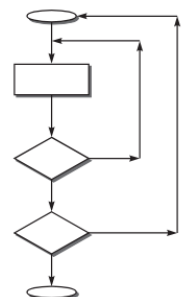
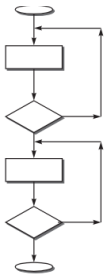


Figure 5.9 (b)





Concatenated loops. The loops in a program may be concatenated (Fig. 5.11). Two loops are concatenated if it is possible to reach one after exiting the other, while still on a path from entry to exit. If the two loops are not on the same path, then they are not concatenated. The two loops on the same path may or may not be independent. If the loop control variable for one loop is used for another loop, then they are concatenated, but nested loops should be treated like nested only.

Unstructured loops. This type of loops is really impractical to test and they must be redesigned or at least converted into simple or concatenated loops.

Data flow testing

Data flow testing is a white-box testing technique that can be used to detect improper use of data values due to coding errors. Errors may be unintentionally introduced in a program by programmers. For instance, a programmer might use a variable without defining it. Moreover, he may define a variable, but not initialize it and then use that variable in a predicate. For example,

```
int a;
if(a == 67) { }
```

In this way, data flow testing gives a chance to look out for inappropriate data definition, its use in predicates, computations, and termination. It identifies potential bugs by examining the patterns in which that piece of data is used. For example, if out-of-scope data is being used in a computation, then it is a bug. There may be several patterns like this which indicate data anomalies.

To examine the patterns, the control flow graph of a program is used. This test strategy selects the paths in the module's control flow such that various sequences of data objects can be chosen. The major focus is on the points at which the data receives values and the places at which the data initialized has been referenced. Thus, we have to choose enough paths in the control flow to ensure that every data is initialized before use and all the defined data have been used somewhere. Data flow testing closely examines the state of the data in the control flow graph, resulting in a richer test suite than the one obtained from control flow graph based path testing strategies like branch coverage, all statement coverage, etc.

Mutation testing

Mutation testing is the process of mutating some segment of code (putting some error in the code) and then testing this mutated code with some test data. If the test data is able to detect the mutations in the code, then the test data is quite good, otherwise we must focus on the quality of test data. Therefore, mutation testing helps a user create test data by interacting with the user to iteratively strengthen the quality of test data.

During mutation testing, faults are introduced into a program by creating many versions of the program, each of which contains one fault. Test data are used to execute these faulty programs with the goal of causing each faulty program to fail. Faulty programs are called mutants of the original program and a mutant is said to be killed when a test case causes it to fail. When this happens, the mutant is considered dead and no longer needs to remain in the testing process, since the faults represented by that mutant have been detected, and more importantly, it has satisfied its requirement of identifying a useful test case. Thus, the main objective is to select efficient test data which have error-detection power. The criterion for this test data is to differentiate the initial program from the mutant. This distinguish-ability between the initial program and its mutant will be based on test results.

Mutation testing process

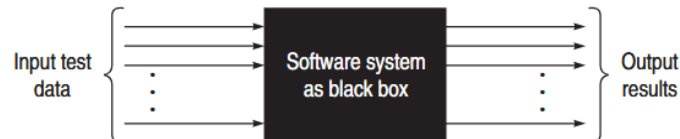
1. Construct the mutants of a test program.
2. Add test cases to the mutation system and check the output of the program on each test case to see if it is correct.
3. If the output is incorrect, a fault has been found and the program must be modified and the process restarted.
4. If the output is correct, that test case is executed against each live mutant.
5. If the output of a mutant differs from that of the original program on the same test case, the mutant is assumed to be incorrect and is killed.
6. After each test case has been executed against each live mutant, each remaining mutant falls into one of the following two categories:
 - a. One, the mutant is functionally equivalent to the original program. An equivalent mutant always produces the same output as the original program, so no test case can kill it.
 - b. Two, the mutant is killable, but the set of test cases is insufficient to kill it. In this case, new test cases

need to be created, and the process iterates until the test set is strong enough to satisfy the tester. The mutation score for a set of test data is the percentage of non-equivalent mutants killed by that data. If the mutation score is 100%, then the test data is called mutation-adequate.

Black box testing

Black-box technique is one of the major techniques in dynamic testing for designing effective test cases. This technique considers only the functional requirements of the software or module. In other words, the structure or logic of the software is not considered. Therefore, this is also known as functional testing. The software system is considered as a black box, taking no notice of its internal structure, so it is also called as black-box testing technique.

It is obvious that in black-box technique, test cases are designed based on functional specifications. Input test data is given to the system, which is a black box to the tester, and results are checked against expected outputs after executing the software, as shown below.



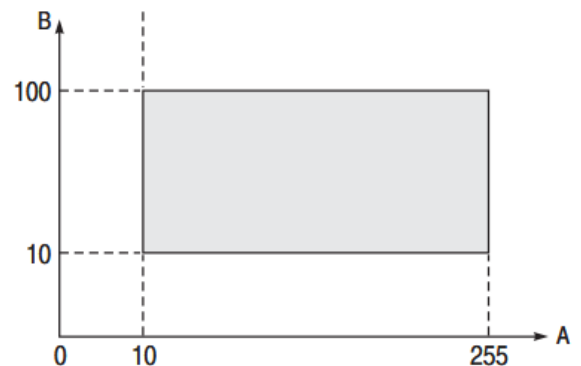
Black-box testing attempts to find errors in the following categories:

- To test the modules independently
- To test the functional validity of the software so that incorrect or missing functions can be recognized
- To look for interface errors
- To test the system behaviour and check its performance
- To test the maximum load or stress on the system
- To test the software such that the user/customer accepts the system within defined acceptable limits

There are various methods to test a software product using black-box techniques. One method chooses the boundary values of the variables, another makes equivalence classes so that only one test case in that class is chosen and executed. Some methods use the state diagrams of the system to form the black-box test cases, while a few other methods use table structure to organize the test cases. It does not mean that one can pick any of these methods for testing the software. Sometimes a combination of methods is employed for rigorous testing. The objective of any test case is to have maximum coverage and capability to discover more and more errors.

Boundary Value Analysis (BVA)

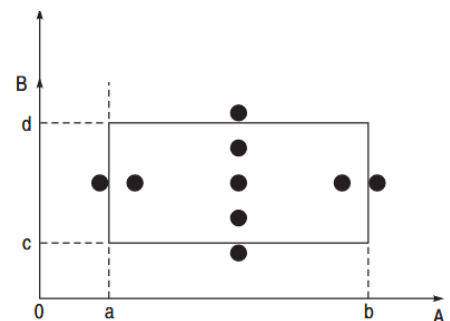
An effective test case design requires test cases to be designed such that they maximize the probability of finding errors. BVA technique addresses this issue. With the experience of the testing team, it has been observed that test cases designed with boundary input values have a high chance to find errors. It means that most of the failures crop up due to boundary values. BVA is considered a technique that uncovers the bugs at the boundary of input values. Here, boundary means the maximum or minimum value taken by the input domain. For example, if A is an integer between 10 and 255, then boundary checking can be on 10(9,10,11) and on 255(256,255,254). Similarly, B is another integer variable between 10 and 100, then boundary checking can be on 10(9,10,11) and 100(99,100,101), as shown below.



Boundary Value Checking (BVC): In this method, the test cases are designed by holding one variable at its extreme value and other variables at their nominal values in the input domain.

The variable at its extreme value can be selected at:

- Minimum value (Min)
- Value just above the minimum value (Min +)
- Maximum value (Max)
- Value just below the maximum value (Max -)



Let us take the example of two variables, A and B. If we consider all the above combinations with nominal values, then following test cases can be designed:

- | | |
|---------------|----------------|
| 1. Anom, Bmin | 2. Anom, Bmin+ |
| 3. Anom, Bmax | 4. Anom, Bmax- |
| 5. Amin, Bnom | 6. Amin+, Bnom |
| 7. Amax, Bnom | 8. Amax-, Bnom |
| 9. Anom, Bnom | |

It can be generalized that for n variables in a module, $4n + 1$ test cases can be designed with the boundary value checking method.

Robustness Testing Method

The idea of BVC can be extended such that boundary values are exceeded as:

- A value just greater than the Maximum value (Max +)
- A value just less than Minimum value (Min -)

When test cases are designed considering the above points in addition to BVC, it is called robustness testing.

Let us take the previous example again. Add the following test cases to the list of 9 test cases designed in BVC:

- | | |
|-----------------|-----------------|
| 10. Amax+, Bnom | 11. Amin-, Bnom |
| 12. Anom, Bmax+ | 13. Anom, Bmin- |

It can be generalized that for n input variables in a module, $6n + 1$ test cases can be designed with robustness testing.

Worst-case Testing Method

We can again extend the concept of BVC by assuming more than one variable on the boundary. It is called the worst-case testing method.

Again, take the previous example of two variables, A and B. We can add the following test cases to the list of 9 test cases designed in BVC as:

- | | |
|------------------|-------------------|
| 10. Amin , Bmin | 11. Amin+ , Bmin |
| 12. Amin , Bmin+ | 13. Amin+ , Bmin+ |
| 14. Amax, Bmin | 15. Amax-, Bmin |
| 16. Amax, Bmin+ | 17. Amax-, Bmin+ |
| 18. Amin , Bmax | 19. Amin+ , Bmax |
| 20. Amin , Bmax- | 21. Amin+ , Bmax- |
| 22. Amax, Bmax | 23. Amax-, Bmax |
| 24. Amax, Bmax- | 25. Amax-, Bmax- |

It can be generalized that for n input variables in a module, $5n$ test cases can be designed with worst-case testing.

BVA is applicable when the module to be tested is a function of several independent variables. This method becomes important for physical quantities where boundary condition checking is crucial. For example, systems having requirements of minimum and maximum temperature, pressure or speed, etc. However, it is not useful for Boolean variables.

Equivalence class testing

We know that the input domain for testing is too large to test every input. So we can divide or partition the input domain based on a common feature or a class of data. Equivalence partitioning is a method for deriving test cases wherein classes of input conditions called equivalence classes are identified such that each member of the class causes the same kind of processing and output to occur. Thus, instead of testing every input, only one test case from each partitioned class can be executed. It means only one test case in the equivalence class will be sufficient to find errors. This test case will have a representative value of a class which is equivalent to a test case containing any other value in the same class. If one test case in an equivalence class detects a bug, all other test cases in that class have the same probability of finding bugs. Therefore, instead of taking every value in one domain, only one test case is chosen from one class. In this way, testing covers the whole input domain, thereby reducing the total number of test cases. In fact, it is an attempt to get a good hit rate to find maximum errors with the smallest number of test cases.

- Equivalence partitioning method for designing test cases has the following goals:
- Completeness. Without executing all the test cases, we strive to touch the completeness of testing domain.
- Non-redundancy. When the test cases are executed having inputs from the same class, then there is redundancy

in executing the test cases. Time and resources are wasted in executing these redundant test cases, as they explore the same type of bug. Thus, the goal of equivalence partitioning method is to reduce these redundant test cases.

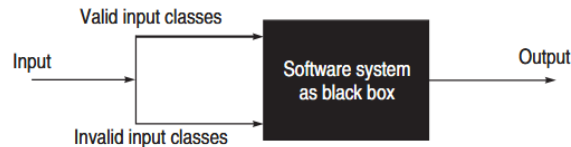
To use equivalence partitioning, one needs to perform two steps:

1. Identify equivalence classes
2. Design test cases

Identification of Equivalent Classes: How do we partition the whole input domain? Different equivalence classes are formed by grouping inputs for which the behaviour pattern of the module is similar. The rationale of forming equivalence classes like this is the assumption that if the specifications require exactly the same behaviour for each element in a class of values, then the program is likely to be constructed such that it either succeeds or fails for each value in that class. For example, the specifications of a module that determines the absolute value for integers specify different behaviour patterns for positive and negative integers. In this case, we will form two classes: one consisting of positive integers and another consisting of negative integers. Two types of classes can always be identified as discussed below:

- Valid equivalence classes. These classes consider valid inputs to the program.
- Invalid equivalence classes. One must not be restricted to valid inputs only.

We should also consider invalid inputs that will generate error conditions or unexpected behaviour of the program, as shown below.



Identifying Test Cases:

A few guidelines are given below to identify test cases through generated equivalence classes:

- Assign a unique identification number to each equivalence class.
- Write a new test case covering as many of the uncovered valid equivalence classes as possible, until all valid equivalence classes have been covered by test cases.
- Write a test case that covers one, and only one, of the uncovered invalid equivalence classes, until all invalid equivalence classes have been covered by test cases. The reason that invalid cases are covered by individual test cases is that certain erroneous-input checks mask or supersede other erroneous-input checks. For instance, if the specification states ‘Enter type of toys (Automatic, Mechanical, Soft toy) and amount (1–10000)’, the test case [ABC 0] expresses two error (invalid inputs) conditions (invalid toy type and invalid amount) will not demonstrate the invalid amount test case, hence the program may produce an output ‘ABC is unknown toy type’ and not bother to examine the remainder of the input.

Remember that there may be many possible solutions for one problem in this technique, depending on the criteria chosen for partitioning the test domain.

Decision-table based testing

Boundary value analysis and equivalence class partitioning methods do not consider combinations of input conditions. These consider each input separately. There may be some critical behaviour to be tested when some combinations of input conditions are considered.

Decision table is another useful method to represent the information in a tabular method. It has the specialty to consider complex combinations of input conditions and resulting actions. Decision tables obtain their power from logical expressions. Each operand or variable in a logical expression takes on the value, TRUE or FALSE.

ENTRY						
Condition Stub		Rule 1	Rule 2	Rule 3	Rule 4	...
	C1	True	True	False	I	
	C2	False	True	False	True	
	C3	True	True	True	I	
Action Stub	A1		X			
	A2	X			X	
	A3			X		

A decision table is formed with the following components:

Condition stub. It is a list of input conditions for which the complex combination is made.

Action stub. It is a list of resulting actions which will be performed if a combination of input conditions is satisfied.

Condition entry. It is a specific entry in the table corresponding to input conditions mentioned in the condition stub. When we enter TRUE or FALSE for all input conditions for a particular combination, then it is called a Rule. Thus, a rule defines which combination of conditions produces the resulting action. When the condition entry takes only two values—TRUE or FALSE, then it is called Limited Entry Decision Table. When the condition entry takes several values, then it is called Extended Entry Decision Table. In the limited entry decision table, condition entry, which has no effect whether it is True or False, is called a Don't-Care state or immaterial state (represented by I). The state of a don't-care condition does not affect the resulting action.

Action entry. It is the entry in the table for the resulting action to be performed when one rule (which is a combination of input conditions) is satisfied. 'X' denotes the action entry in the table. The guidelines to develop a decision table for a problem are discussed below:

- List all actions that can be associated with a specific procedure (or module).
- List all conditions (or decisions made) during execution of the procedure.
- Associate specific sets of conditions with specific actions, eliminating impossible combinations of conditions; alternatively, develop every possible permutation of conditions.
- Define rules by indicating what action occurs for a set of conditions.

Test Case Design using Decision Table: Following interpretations should be done:

- Interpret condition stubs as the inputs for the test case.
- Interpret action stubs as the expected output for the test case.
- Rule, which is the combination of input conditions, becomes the test case itself.
- If there are k rules over n binary conditions, there are at least k test cases and at the most 2^n test cases.

Usability testing

This type of system testing is related to a system's presentation rather than its functionality. System testing can be performed to find human-factors or usability problems on the system. The idea is to adapt the software to users' actual work styles rather than forcing the users to adapt according to the software. Thus, the goal of usability testing is to verify that intended users of the system are able to interact properly with the system while having a positive and convenient experience. Usability testing identifies discrepancies between the user interfaces of a product and the human engineering requirements of its potential users.

What the user wants or expects from the system can be determined using several ways:

- Area experts. The usability problems or expectations can be best understood by the subject or area experts who have worked for years in the same area. They analyse the system's specific requirements from the user's perspective and provide valuable suggestions.
- Group meetings. Group meeting is an interesting idea to elicit usability requirements from the user. These meetings result in potential customers' comments on what they would like to see in an interface.
- Surveys. Surveys are another medium to interact with the user. It can also yield valuable information about how potential customers would use a software product to accomplish their tasks.
- Analyse similar products. We can also analyse the experiences in similar kinds of projects done previously and use it in the current project. This study will also give us clues regarding usability requirements.

Usability characteristics against which testing is conducted are discussed below:

- Ease of use. The users enter, navigate, and exit with relative ease. Each user interface must be tailored to the intelligence, educational background, and environmental pressures of the end-user.
- Interface steps. User interface steps should not be misleading. The steps should also not be very complex to understand either.
- Response time. The time taken in responding to the user should not be so high that the user is frustrated or will move to some other option in the interface.
- Help system. A good user interface provides help to the user at every step. The help documentation should not be redundant; it should be very precise and easily understood by every user of the system.
- Error messages. For every exception in the system, there must be an error message in text form so that users can understand what has happened in the system. Error messages should be clear and meaningful.

An effective tool in the development of a usable application is the user-interface prototype. This kind of prototype allows interaction between potential users, requirements personnel, and developers to determine the best approach to the

system's interface. Prototypes are far superior because they are interactive and provide a more realistic preview of what the system will look like.

Accessibility testing:

Accessibility testing is a specialized type of usability testing that focuses on ensuring websites and applications can be properly accessed by people with disabilities. The primary goal is to create an inclusive digital experience by eliminating obstacles that limit usability, ensuring that everyone can navigate, interact with, and benefit from digital content.

Types of Accessibility Testing

Manual Testing: Manual accessibility testing involves examining a website or application's conformance with relevant web accessibility standards. Testers manually check various elements using assistive technologies like screen readers to understand the user experience. This approach includes:

- Testing brightness and color contrast adjustments
- Checking sound performance for hearing-impaired users
- Verifying caption visibility and expressiveness
- Testing font size modifications
- Using high contrast mode
- Turning off CSS to check background colors and text presentation

Automated Testing: This approach relies on automated web accessibility testing tools to examine a website's conformance with accessibility guidelines. Automated tools can quickly scan web pages to identify common accessibility issues, providing efficient ways to detect problems at scale.

Screen Reader Testing: Testers specifically focus on testing screen readers used by visually impaired users, checking proper navigation, labeling, and all elements while using a screen reader.

Keyboard Navigation Testing: Applications are tested for navigation using only a keyboard, replacing the mouse for users with physical disabilities. Testers verify tab order, keyboard shortcuts, and the ability to access all interactive elements.

Key Testing Areas by Disability Category

Visual Impairments: For users with blindness, low vision, or color blindness, testing focuses on:

Screen reader compatibility:

- Color contrast ratios between text and background
- Text resizing capability to 200% without loss of content or function

Physical Disabilities: Testing ensures websites can be operated fully using only a keyboard for users who have difficulty using a computer mouse.

Hearing Impairments: Testing addresses obstacles faced by users with deafness or other hearing impairments when accessing spoken communication or digital media.

Cognitive Impairments: Testing considers users who may have difficulties with memory, concentration, or decision-making.

Regression Testing:

Regression testing is a software testing process that ensures recent code changes, such as new features or bug fixes, do not negatively impact previously working functionality. It involves re-running established functional and non-functional test cases to verify that the software's existing features still perform as expected. This process helps catch new bugs (regressions) introduced by the changes, ensuring the overall stability and reliability of the software.

Objectives of Regression Testing

Quality Assurance and Defect Detection

- Detecting unintentional defects by identifying bugs that may have been introduced due to recent code changes
- Validating changes to confirm that recent code changes have not introduced new defects or broken existing features
- Ensuring early defect identification, preventing additional costs related to bug fixing in the future by

identifying defects earlier in the software development lifecycle

System Stability and Performance

- Ensuring existing functionality remains intact after modifications, updates, or new feature additions
- Verifying system stability and maintaining software quality throughout the development lifecycle
- Maintaining overall quality and performance of the application across releases

Risk Management and Reliability

- Preventing regression bugs from reaching production environments where they could impact end users
- Reducing risks by minimizing potential risks associated with changes, safeguarding against adverse effects on existing functionalities
- Improving software reliability by testing the software under a variety of conditions to identify and fix any defects that may cause the software to fail

Development and Deployment Support

- Establishing confidence in software releases by validating that changes don't break previously working features
- Supporting continuous delivery by enabling seamless and confident deployment through verification of core features every time
- Facilitating continuous testing, enhancing the overall quality and stability of the software

Need for Regression Testing

Technical Necessity

- Code interdependencies: Modern software systems have complex interdependencies where changes in one module can unexpectedly affect other parts of the system
- Unintended side effects: Even the slightest change in the code can have disastrous consequences, particularly if the product is already in use by customers
- System complexity: As applications grow in complexity, the potential for unexpected interactions between components increases significantly

Business and Development Requirements

- Continuous development: As new features are constantly added and changes are made throughout the software development lifecycle, regression testing ensures stability
- Agile development support: In Agile environments with short sprint cycles, developers introduce specific features in each cycle, requiring testing to ensure new features don't conflict with previously written code
- Rapid development cycles: Supports faster release cycles without compromising reliability, allowing for frequent code updates while maintaining quality

Cost and Time Benefits

- Early problem detection: Saves time and money by preventing defects from being found later in the development process
- Reduced debugging costs: By identifying and fixing defects early on, regression testing helps reduce the cost of fixing defects later in development
- Quality maintenance: Helps ensure a consistent user experience across versions, maintaining customer satisfaction

Types of Regression Testing

Based on Scope and Coverage

Complete Regression Testing

- Involves re-testing the entire application when significant multiple modifications are made to existing code
- Most comprehensive but time-consuming and resource-intensive
- Very useful when multiple changes have been done in the existing code and when changes have certain impact on software's root code
- Typically used for major releases or when establishing a new baseline of functionality

Partial Regression Testing

- Tests specific parts of the system that have been changed, along with directly related components
- More suitable for large-scale applications with numerous test scripts
- Balances risk against time and resources required for testing

Selective Regression Testing

- Uses a subset of existing test cases to reduce time and cost
- Selects specific test cases strategically based on risk analysis and impact
- Focuses on areas most likely to be affected by the changes
- Objective: To find the dependencies between a test case and the program entities it covers
- Viewed as a middle ground between corrective and retest-all approaches

Based on Code Modifications

Corrective Regression Testing

- Used when no changes are made to source code or product specifications
- Reuses existing test cases, requiring minimal effort and cost
- Tests overall functionality to determine if existing test cases can be reused
- Appreciated for its simplicity as it requires less effort and reuses existing test cases when no major specification changes have been made

Progressive Regression Testing

- Applied when specifications are modified and new test cases must be created
- Used during application updates to ensure existing features remain intact
- Recommended when developing new test cases and allows testers to perform required steps within modified program versions
- Works effectively when there are changes in software specifications and new test cases need to be designed

Retest-All Regression Testing

- Re-executes all test scenarios to compare results with previous testing
- Very time-consuming because it reuses all tests, resulting in execution of avoidable test cases
- Generally avoided unless the application is small due to significant effort required
- Not recommended for small modifications due to time constraints and unnecessary test execution

Based on Testing Level

Unit Regression Testing

- Focuses on testing individual units or components in isolation
- Conducted at the unit testing stage to detect errors early
- Ideal for testing small-scale changes like individual functions or methods

Based on Execution Method

Automated Regression Testing

- Uses automated tools to run predefined test cases
- Perfect for repetitive tasks and large-scale applications
- Saves time and reduces human error for frequent releases
- Popular tools include: Selenium, Cypress, Appium, Playwright, Puppeteer, and BrowserStack

Manual Regression Testing

- Involves human testers running test cases manually
- More flexible for scenarios difficult to automate, such as UI testing
- Useful when exploratory testing or human judgment is required